

BC7215A Offline Air Conditioner Control Library

Linux(C++) Demo

and

C++ Library

User Manual

- Uses a normal Linux terminal as the interactive user interface.
- Demonstrates pairing, air-conditioner control, saving/loading pairing data, predefined protocols, match-next, and IR command parsing.

Preface

This demo is a Linux/POSIX port of the ESP-IDF C++/FreeRTOS demo. It relies on the low level C library wrapped them into 2 C++ libraries:

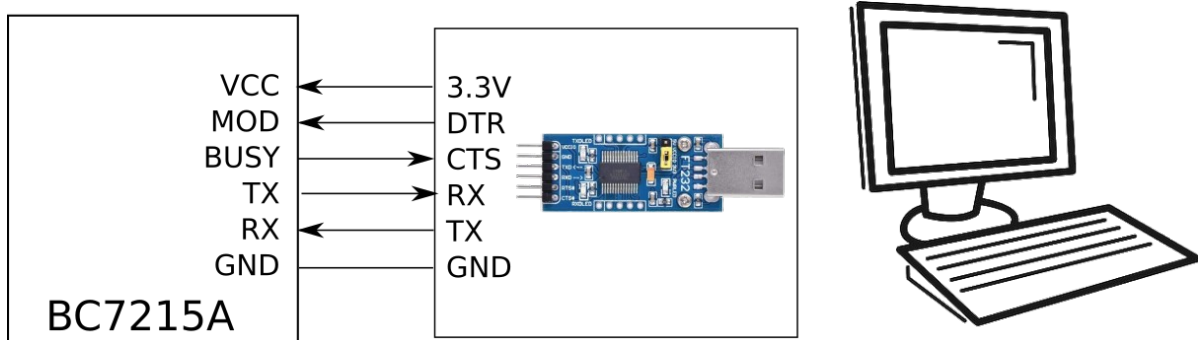
- **bc7215::BC7215**: low-level BC7215/BC7215A driver wrapper. It manages serial communication, CTS hardware flow control, DTR-based MOD control, and the RX thread.
- **bc7215::BC7215AC**: high-level air-conditioner control wrapper. It uses `BC7215` class internally and provides simplified functions for pairing, control, parsing, saving, and loading.

This version is intended to run on both x86 Linux computers and embedded Linux boards. It uses only `gcc/g++`, `libc/libstdc++`, and POSIX APIs. No third-party library is required.

Hardware Connection and Configuration

Hardware connection scenarios can be classified into 5 cases:

1. Desktop host connected to the BC7215 circuit using a USB-to-TTL converter with CTS/DTR signals.
2. Embedded host connected to the BC7215 circuit using a USB-to-TTL converter with CTS/DTR signals.
3. Embedded Linux board connected to the BC7215 circuit using an onboard UART serial port with CTS/DTR signals.
4. Embedded Linux board connected to the BC7215 circuit using a UART serial port with CTS signals but without DTR.
5. Embedded Linux board connected to the BC7215 circuit using a UART serial port with only TX/RX signals (no CTS or DTR signals).



For the first three scenarios, the demonstration software can be run directly without any modifications. Many embedded Linux systems that support USB interfaces, such as Raspberry Pi, come with built-in drivers for common USB-to-TTL converters, enabling plug-and-play functionality.

In the scenario where the DTR signal is missing but the CTS signal is present, the function of the DTR signal is to control the logic level of the BC7215's MOD pin to switch its input/output state.

Therefore, the most convenient approach is to control a GPIO pin to substitute for the DTR signal. Users only need to modify the contents of the following three functions within the BC7215 class in the example code to perform the corresponding GPIO operations:

```
static void mod_set_high_cb();
static void mod_set_low_cb();
static uint8_t read_mod_cb();
```

If both DTR and CTS signals are missing, the configuration becomes slightly more complex. Because the CTS signal is used for hardware flow control, the absence of CTS requires providing an additional input GPIO pin to connect to the BC7215's BUSY signal, along with the following software modifications:

1. Modify the serial port settings to disable hardware flow control.
2. Modify the bc7215_lib_config.h file to disable hardware flow control, and then provide a single-byte UART transmit function. This function must block and only return after the data has been completely transmitted from the serial port.
3. Register this transmit function and the BUSY pin read function to the underlying driver by passing them into the following two functions inside the begin() function:

```
bc7215_config_uart_send_byte();
bc7215_config_read_busy_funct();
```

Linux serial permission

Check the serial device name:

```
ls /dev/ttyUSB*
ls /dev/ttyACM*
ls /dev/ttyS*
ls /dev/ttyAMA*
```

Examples:

/dev/ttyUSB0	USB-to-TTL adapter
/dev/ttyS1	SoC UART or PC serial port
/dev/ttyAMA1	Raspberry Pi / ARM SoC UART

If permission is denied, either run with sudo for testing:

```
sudo ./bc7215_ac_demo --dev /dev/ttyUSB0
```

or add the user to the serial-access group. On many Linux distributions this group is dialout:

```
sudo usermod -aG dialout $USER
```

Log out and log in again after changing group membership.

Build tools

Install a C/C++ compiler and make. For example, on Debian/Ubuntu:

```
sudo apt update
```

```
sudo apt install build-essential
```

For embedded Linux cross-compilation, install the corresponding cross toolchain, for example:

```
sudo apt install gcc-arm-linux-gnueabi g++-arm-linux-gnueabi
```

Directory Structure

```
Project_Name
├── bc7215_ac_lib
│   ├── bc7215_types.h
│   ├── bc7215_lib.h
│   ├── bc7215_ac_lib.h
│   ├── bc7215_lib.c
│   ├── bc7215_ac_lib.c
│   └── 其他库文件
└── examples
    ├── Linux
    │   ├── Makefile
    │   ├── include
    │   │   ├── bc7215_lib_config.h
    │   │   ├── bc7215.hpp
    │   │   ├── bc7215ac.hpp
    │   │   └── console_line.hpp
    │   └── src
    │       ├── bc7215.cpp
    │       ├── bc7215ac.cpp
    │       ├── console_line.cpp
    │       └── main.cpp
```

Usage

Build

From the project root:

```
cd examples/Linux
make
```

The output executable is:

```
examples/Linux/bc7215_ac_demo
```

Clean the build output:

```
make clean
```

Cross-compile example:

```
make CROSS_COMPILE=arm-linux-gnueabihf-
```

or explicitly:

```
make CC=arm-linux-gnueabihf-gcc CXX=arm-linux-gnueabihf-g++
```

Run

Basic usage:

```
./bc7215_ac_demo --dev /dev/ttyUSB0
```

Other examples:

```
./bc7215_ac_demo --dev /dev/ttyS1
./bc7215_ac_demo --dev /dev/ttyAMA1
./bc7215_ac_demo --dev /dev/ttyUSB0 --store ./ac_saved.bin
./bc7215_ac_demo --dev /dev/ttyUSB0 --mod-reverse
```

Command-line options:

Option	Description
--dev <device>	Select the Linux serial device. Example: /dev/ttyUSB0.
--store <file>	Select the file used to save and load pairing data.
--mod-reverse	Reverse the default DTR/MOD polarity. Use this only if the measured MOD level is opposite.
--help	Show help.

Console Operation

After the example program starts, it first asks the user to select whether the controlled air conditioner uses Celsius or Fahrenheit, because air conditioners using different temperature units need to call different functions.

```

System initialized.
Serial device: /dev/ttyUSB1
MOD control: DTR (default polarity)
Store file: bc7215_ac_saved.bin
All menu commands take effect only after Enter. Backspace editing is handled by the terminal.
Please choose your AC's temperature unit, then press Enter:
  1. Celsius
  2. Fahrenheit
> 1
AC has been set to Celsius.

*****
* Welcome to BC7215A Universal AC Controller Demo *
*****
AC Library Version: V7.1 build 2619
Current AC control library status: Not initialized (must be paired before use)
Temperature Unit: Celsius
Please select, then press Enter:
  1. Pairing with AC
  2. Control air conditioner
  3. Save pairing data
  4. Read saved data and pair with it
  5. Try next match
  6. Load predefined protocol
  7. Parse IR signal
  8. Set AC temp unit
> █

```

After that, the main menu is displayed in the terminal. The main menu shows the AC library version and the current status of the air conditioner code library, that is, whether it has been initialized (paired) with the air conditioner.

After the main menu is displayed, enter the menu number in the terminal and press Enter to enter the corresponding function.

1. Initialization (Pairing)

After the program starts, it should first complete pairing (initialization) with the air conditioner before any other functions can be operated. The operation process is basically the same as that of the DEMO software on the PC. Users can refer to the section about the DEMO software in the BC7215A DEMO board manual. In this example, IR signal capture and "initialization" are combined into one "pairing" operation.

```

Please select, then press Enter:
1. Pairing with AC
2. Control air conditioner
3. Save pairing data
4. Read saved data and pair with it
5. Try next match
6. Load predefined protocol
7. Parse IR signal
8. Set AC temp unit

> 1

Now performing IR AC pairing.
Please set AC remote to 25C, Cool, Fan, then press Enter.
>
Now aim at IR receiver and press Fan button on remote.
Will automatically proceed after receiving signal...
Received data: 85 19 12 FF FF 52 7A E6 ED 00 00 AD
AC control library initialization using received data SUCCESS.
Press Enter to return to the main menu.
>

*****
* Welcome to BC7215A Universal AC Controller Demo *
*****
AC Library Version: V7.1 build 2619
Current AC control library status: ***INITIALIZED***
Temperature Unit: Celsius
Please select, then press Enter:
1. Pairing with AC
2. Control air conditioner
3. Save pairing data
4. Read saved data and pair with it
5. Try next match
6. Load predefined protocol
7. Parse IR signal
8. Set AC temp unit

> █

```

After a menu operation is selected, the next operation prompt is displayed. The figure above shows the pairing process.

2. 控制空调

2. Air Conditioner Control

After initialization is completed, the air conditioner can be controlled. Select '2' in the main menu to enter the air conditioner control submenu.

The air conditioner control menu has four items. Select '1' to enter the air conditioner parameter control page, '2' or '3' to control power on/off, and '4' to return to the main menu.

On the parameter control page, enter four parameters in order: "temperature, mode, fan speed, remote-control key". Separate them with commas. Partial input is supported. For example, if you only need to change the temperature, you can enter only the temperature; in this case the other settings remain unchanged.

Enter the English word "exit" to exit air conditioner parameter control and return to the upper-level menu.

The valid input ranges are displayed when entering the parameter control page. Inputs outside the valid range are treated as keeping the corresponding setting unchanged. After a setting value is entered, the terminal displays the received setting, immediately starts IR transmission, and displays the transmitted data. After transmission is complete, it waits for the next input.

See the figure below for the user interface:

```

AC Library Version: V7.1 build 2619
Current AC control library status: ***INITIALIZED***
Temperature Unit: Celsius
Please select, then press Enter:
1. Pairing with AC
2. Control air conditioner
3. Save pairing data
4. Read saved data and pair with it
5. Try next match
6. Load predefined protocol
7. Parse IR signal
8. Set AC temp unit

> 2

AC Control, please select, then press Enter:
1. Set AC parameters
2. Power on
3. Power off
4. Return to upper menu

> 1
*** AC parameter adjustment ***
Format: temperature,mode,fan,key e.g. 24,1,2,0
Fewer parameters are allowed, e.g. 18,2 means 18C Heating, fan/key unchanged.
Temperature(C)  Mode   Fan    Key
Range: 16-30   0-Auto 0-Auto 0-Temp+
               1-Cool 1-Low  1-Temp-
               2-Heat 2-Med  2-Mode
               3-Dry  3-High 3-Fan
               4-Fan
Values outside the range mean keep current state. Enter 'exit' to return.

> 23, 2
Sending command to set AC to: 23C, Mode: Heat, Fan Speed: Keep, Key Press: Keep
Sending data: 85 D9 62 FF FF 42 7A 26 9D 00 00 BD
Transmission complete.

Please continue input. Enter 'exit' to return.
> 

```

3. IR Command Parsing

After pairing succeeds, IR commands can also be parsed. Note that parsing is valid only for the paired air conditioner remote control. Select the following item in the main menu:

7 - Parse IR Signal

to enter parsing mode. After parsing mode is entered, the BC7215A switches to receiving mode. At this time, if you press a remote-control key pointing it at the IR receiver, the program parses and prints the four parameters in the command: temperature, mode, fan speed, and power status.

In parsing mode, press Enter to return to the main menu.


```

2. Power on
3. Power off
4. Return to upper menu

> 4

*****
* Welcome to BC7215A Universal AC Controller Demo *
*****
AC Library Version: V7.1 build 2619
Current AC control library status: ***INITIALIZED***
Temperature Unit: Celsius
Please select, then press Enter:
1. Pairing with AC
2. Control air conditioner
3. Save pairing data
4. Read saved data and pair with it
5. Try next match
6. Load predefined protocol
7. Parse IR signal
8. Set AC temp unit

> 7

Receiving IR signal and parsing Temperature, Mode, Fan Speed, and Power.
BC7215A is now in RX mode. Type anything and press Enter to exit.
Parsing Result: Temp: 25C, Mode: Dry, Fan Speed: Auto, Power: ON
Parsing Result: Temp: 25C, Mode: Heat, Fan Speed: High, Power: ON
Parsing Result: Temp: n/a, Mode: Fan, Fan Speed: High, Power: ON
Parsing Result: Temp: 25C, Mode: Auto, Fan Speed: Auto, Power: ON
Parsing Result: Temp: 25C, Mode: Cool, Fan Speed: High, Power: ON
Parsing Result: Temp: 24C, Mode: Cool, Fan Speed: High, Power: ON
Parsing Result: Temp: 23C, Mode: Cool, Fan Speed: High, Power: ON
Parsing Result: Temp: 22C, Mode: Cool, Fan Speed: High, Power: ON
Parsing Result: Temp: 21C, Mode: Cool, Fan Speed: High, Power: ON
Parsing Result: Temp: 21C, Mode: Cool, Fan Speed: Auto, Power: ON
Parsing Result: Temp: 21C, Mode: Cool, Fan Speed: Low, Power: ON
Parsing Result: Temp: 21C, Mode: Cool, Fan Speed: Low, Power: OFF

```

4. Others

In this example, information is saved directly as a binary file to ensure maximum platform compatibility and avoid dependencies on external libraries. The filename and path for the saved file can be specified via the startup command.

For the purpose of the operations for finding the next match and loading a predefined protocol, refer to the BC7215A Offline Air Conditioner IR Control Library User Manual.

Using the C++ Classes

The C-language library is wrapped into C++ classes, all placed in the "bc7215" namespace.

bc7215::BC7215 Class

This is the underlying driver wrapper class. It integrates Linux serial port operations, MOD control via DTR, CTS flow control, and an RX polling thread.

The BC7215 class provides the exact same functions as the C driver; its public functions map directly to the C-language version and offer identical functionality. However, it provides overloads for the following functions, allowing references to bc7215DataMaxPkt_t and bc7215FormatPkt_t data types to be passed directly. This eliminates the tedious process of pointer type casting:

```

uint8_t get_data(bc7215DataMaxPkt_t& target);
uint8_t get_format(bc7215FormatPkt_t& target);
void ir_tx(const bc7215DataMaxPkt_t& source);

```

bc7215::BC7215AC class

This class wraps the C-language air conditioner code library and provides easier-to-use functions. For example, the sampling-and-pairing operation is simplified into calls to `start_capture()` and `init()`, with the specific execution logic completed inside the class. The BC7215 class is one of its members, so users do not need to define a separate BC7215 class variable during use.

Public Member Variables:

The BC7215AC class provides several externally accessible variables for convenient use:

```
bool init_ok;           // Whether initialization (pairing) has been completed
uint8_t sample_count;   // Number of sampled data segments, up to 4
uint8_t sample_status[4]; // Status of each segment
bc7215DataMaxPkt_t sample_data[4]; // Data packet of each segment
bc7215FormatPkt_t sample_format[4]; // Format packet of each segment
```

After sampling ends, if users need to display the contents of the received data, they can access these variables directly.

Public Member Functions:

Constructor:

The parameters consist of the serial device name and a flag to invert the DTR polarity. These parameters are passed to the internal BC7215 class member to complete the serial port configuration.

begin() :

Before use, the `begin()` function must be called to complete the necessary initialization. If it fails, a false is returned.

Temperature Unit Functions:

```
void set_fahrenheit();
void set_celsius();
bool is_celsius();
```

Functions for setting and querying the air conditioner temperature unit. If the unit is changed, pairing becomes invalid and pairing must be performed again.

Sampling Functions:

```
void start_capture(uint8_t rx_mode);
void stop_capture();
bool signal_captured();
```

IR signal capture commands. The parameter of the start-capture command is the RX mode after the BC7215 chip enters receiving mode. During pairing, complex mode (`rx_mode = 1`) must be used. During IR parsing, simple mode can be used to reduce data transmission.

The handling of multi-segment data is already included in the functions, so users do not need to care about the processing flow for multi-segment data sampling in C.

Initialization (Pairing) Functions:

```
bool init();
bool init(const bc725DataMaxPkt_t& data, const bc7215FormatPkt_t& format);
```

```
bool match_next();
bool init_predefined(uint8_t index);
```

The operation flow of the initialization (pairing) functions has already been wrapped inside the class. The pairing process is simplified to four function calls: start_capture(1) ==> check signal_captured() ==> stop_capture() ==> init().

For scenarios where saved pairing information is used for initialization at power-on, use the parameterized version:

```
init(const bc725DataMaxPkt_t& data, const bc7215FormatPkt_t& format);
```

When using predefined protocol data for initialization, a single call to init_predefined(uint8_t index) is enough. The parameter is the data index.

The function of match_next() is the same as bc7215_find_next_match() in the C-language version.

Air Conditioner Control Functions:

```
const bc7215DataVarPkt_t* set_to(int temp, int mode, int fan, int key);
const bc7215DataVarPkt_t* on();
const bc7215DataVarPkt_t* off();
```

The wrapped C++ class integrates command data generation from the C-language library with IR data transmission, so users do not need to care about how to process the generated command data and transmit it by IR. All air conditioner control commands are condensed into these three functions. After calling these functions for power on/off or status setting, data transmission is completed automatically.

The class internally caches the last air conditioner status. When the air conditioner is turned on again after being turned off, the previous settings are restored. In parsing mode, each successfully parsed air conditioner setting is also synchronized to the internally cached air conditioner status.

* Most air conditioners do not have a dedicated power-on command. When the air conditioner is off, sending commands such as temperature settings will automatically turn it on.

IR Parsing Function:

```
bool parse(int& temp, int& mode, int& fan, int& power);
```

The parsing function is basically the same as the C-language version, but the output temperature is restored according to the air conditioner temperature unit. That is, when using Celsius, the temp range is 16-30; when using Fahrenheit, the output temp range is 60-88.

Other Functions:

```
uint8_t predefined_count();           // Get the number of predefined protocols
const char* predefined_name(uint8_t index); // Get the predefined protocol
                                           name
bool is_busy();                       // Query whether it is busy; can be used to
                                           check whether IR command transmission is
                                           complete
const bc7215DataVarPkt_t* data_packet(); // Get the base data packet
const bc7215FormatPkt_t* format_packet(); // Get the base format packet
const char* lib_version();             // Query the library version
const BC7215& driver();                // Get the underlying driver; use this function
                                           when direct operation of the underlying driver
                                           library is required
```

When initialization information needs to be saved for use at the next power-on, it can be obtained through the commands for getting the base data packet and base format packet. It is not

recommended to read directly from the public variables after sampling, because there may sometimes be multiple data packets. In addition, after initialization succeeds and before saving, the parsing function should not be executed, because the parsing function replaces the base data packet.